

GEMV 2: Memory DSDs

Contents

- [layout.csl](#)
- [pe_program.csl](#)
- [run.py](#)
- [commands.sh](#)

Continuing on from the previous example, we now extend it by introducing memory Data Structure Descriptors (DSDs), an efficient mechanism for performing operations on entire tensors.

This program creates three one-dimensional memory DSDs for accessing A , b , and y , each of which specifies how to loop over the respective arrays.

b_dsd and y_dsd access the M contiguous elements of b and y , respectively. A_dsd accesses M elements of A , but strided by N elements. Because A is stored in row major format, this means that A_dsd initially accesses the 0th column of A .

We demonstrate here two ways of defining DSDs. For y_dsd , we specify the base memory address ($\&y$) and the number of elements accessed (M). For A_dsd and b_dsd , we demonstrate the use of a `tensor_access` expression. The `tensor_access` field specifies an induction variable, a loop bound, and an affine expression (i.e., a linear function plus a constant) to generate various addresses at runtime.

These DSDs are used by the DSD operations `@fmacs` and `@fadds` to compute $Ax + b$ and store it in y . The `gemv` function first loops over N , with the `@fmacs` in iteration i computing the scalar-vector product of $x[i]$ with column i of A , and incrementing y by that result. The `increment_dsd_offset` operation updates A_dsd by shifting its access by one element. This causes A_dsd to access the next column of A . After the loop, y is incremented by b with the `@fadds` operation, to complete the computation.

Other DSD operations and their associated operand types are described in [Builtins for DSD Operations](#).

Note

See [GEMV Tutorial 2: Memory DSDs](#) for a step-by-step walkthrough of this example.

layout.csl

```

const memcpy = @import_module("<memcpy/get_params>", .{ .width = 1, .height = 1 });

layout {
    @set_rectangle(1, 1);
    @set_tile_code(0, 0, "pe_program.csl", .{ .memcpy_params = memcpy.get_params(0) });

    // export symbol names
    @export_name("y", [*]f32, false);
    @export_name("init_and_compute", fn()void);
}

```

pe_program.csl

```

param memcpy_params;

// memcpy module provides infrastructure for copying data
// and launching functions from the host
const sys_mod = @import_module("<memcpy/memcpy>", memcpy_params);

// Constants defining dimensions of our matrix
const M: i16 = 4;
const N: i16 = 6;

// 48 kB of global memory contain A, x, b, y
var A: [M*N]f32; // A is stored row major

// Initialize x, b, y using builtins
var x = @constants([N]f32, 1.0);
var b = @constants([M]f32, 2.0);
var y = @zeros([M]f32);

// DSDs for accessing A, b, y
// b_dsd uses tensor access expression to specify access to M consecutive elements of b
var b_dsd = @get_dsd(mem1d_dsd, .{ .tensor_access = |i|{M} -> b[i] });
// The above expression is equivalent to:
// var b_dsd = @get_dsd(mem1d_dsd, .{ .base_address = &b, .extent = M });

// y_dsd uses base_address and extent fields to specify access to M consecutive elements of y
var y_dsd = @get_dsd(mem1d_dsd, .{ .base_address = &y, .extent = M });
// The above expression is equivalent to:
// var y_dsd = @get_dsd(mem1d_dsd, .{ .tensor_access = |i|{M} -> y[i] });

// A_dsd accesses column of A
// A_dsd uses tensor access expression to specify access to every Nth element of A
var A_dsd = @get_dsd(mem1d_dsd, .{ .tensor_access = |i|{M} -> A[i*N] });
// The above expression is equivalent to:
// var A_dsd = @get_dsd(mem1d_dsd, .{ .base_address = &A, .extent = M, .stride = N });

// ptr to y will be advertised as symbol to host
const y_ptr: [*]f32 = &y;

```

```

// Initialize A matrix
fn initialize() void {
    // for loop with range syntax
    for (@range(i16, M*N)) |idx| {
        A[idx] = @as(f32, idx);
    }
}

// Compute gemv
fn gemv() void {
    // Loop over all columns of A
    for (@range(u16, N)) |i| {
        // Calculate contribution to A*x from ith column of A, ith elem of x
        @fmacs(y_dsd, y_dsd, A_dsd, x[i]);
        A_dsd = @increment_dsd_offset(A_dsd, 1, f32);
    }
    // Add b to A*x
    @fadds(y_dsd, y_dsd, b_dsd);
}

// Call initialize and gemv functions
fn init_and_compute() void {
    initialize();
    gemv();
    sys_mod.unblock_cmd_stream();
}

comptime {
    @export_symbol(y_ptr, "y");
    @export_symbol(init_and_compute);
}

```

run.py

```
#!/usr/bin/env cs_python

import argparse
import numpy as np

from cerebras.sdk.runtime.sdkruntimepybind import SdkRuntime, MemcpyDataType, MemcpyOrder #
pylint: disable=no-name-in-module

# Read arguments
parser = argparse.ArgumentParser()
parser.add_argument('--name', help="the test compile output dir")
parser.add_argument('--cmaddr', help="IP:port for CS system")
args = parser.parse_args()

# Matrix dimensions
M = 4
N = 6

# Construct A, x, b
A = np.arange(M*N, dtype=np.float32).reshape(M, N)
x = np.full(shape=N, fill_value=1.0, dtype=np.float32)
b = np.full(shape=M, fill_value=2.0, dtype=np.float32)

# Calculate expected y
y_expected = A@x + b

# Construct a runner using SdkRuntime
runner = SdkRuntime(args.name, cmaddr=args.cmaddr)

# Get symbol for copying y result off device
y_symbol = runner.get_id('y')

# Load and run the program
runner.load()
runner.run()

# Launch the init_and_compute function on device
runner.launch('init_and_compute', nonblock=False)

# Copy y back from device
y_result = np.zeros([1*1*M], dtype=np.float32)
runner.memcpy_d2h(y_result, y_symbol, 0, 0, 1, 1, M, streaming=False,
    order=MemcpyOrder.ROW_MAJOR, data_type=MemcpyDataType.MEMCPY_32BIT, nonblock=False)

# Stop the program
runner.stop()

# Ensure that the result matches our expectation
np.testing.assert_allclose(y_result, y_expected, atol=0.01, rtol=0)
print("SUCCESS!")
```

commands.sh

```
#!/usr/bin/env bash
```

```
set -e
```

```
cslc --arch=wse3 ./layout.csl --fabric-dims=8,3 \  
--fabric-offsets=4,1 -o out --memcpy --channels 1  
cs_python run.py --name out
```

© Copyright 2026, Cerebras Systems.

Last updated on Apr 15, 2026, 7:13:14 PM.